TECHNICAL CASE STUDY

Building an AI Restaurant Operations *Partner*

An AI-native platform for independent restaurants — architected and built solo.

DISCIPLINE

Full-stack engineering
AI systems · Multi-tenant SaaS

STACK

FastAPI · Supabase
Claude API · Vercel · Railway

FORMAT

Implementation-level walkthrough
of architecture & AI features

CONTENTS

What's Inside

SECTION 01

Overview

This is a restaurant operations platform built around an AI assistant called *Sage*. It is not a thin wrapper over a chat model.

It is a multi-tenant SaaS application with a Python/FastAPI backend, a Postgres data layer on Supabase with Row-Level Security, a vanilla-JS progressive web app frontend, and a set of AI features that are each grounded in a specific slice of a restaurant's operational data.

The product covers the daily reality of running an independent restaurant: scheduling staff, counting and reordering inventory, costing recipes, parsing supplier invoices, forecasting covers and revenue, tracking review sentiment, and surfacing proactive operational alerts. What ties it together is that every one of these surfaces feeds a shared intelligence layer, and Sage reasons over that layer to give the operator advice that is specific to *their* restaurant — its peak days, its cost drivers, its operator's habits, even its owner's voice.

This document walks through the problem space, the system architecture, each AI feature in implementation-level detail, the data model and multi-tenant security model, the testing and deployment pipeline, and a closing reflection on what was genuinely hard.

SECTION 02

The Problem Space

Independent restaurant operators run on thin margins and almost no back-office staff. The owner is frequently also the GM, the buyer, the scheduler, and the person reading the Google reviews. The information they need to make good decisions exists — in their POS, in supplier invoices, in inventory counts, in review platforms — but it is scattered across systems that do not talk to each other, and none of it is synthesized into "what should I do today."

Enterprise restaurant software exists, but it is priced and scoped for chains with corporate analysts. The independent operator is left with spreadsheets and intuition. The specific failure modes this creates:

- **Food cost creep that nobody catches.** A protein price rises 12% over six weeks; the operator does not notice until the monthly P&L, and even then cannot tell whether the cause is vendor pricing, over-portioning, or waste.
- **Reactive ordering.** Inventory counts happen on a clipboard, get transcribed late or not at all, and reorder decisions are made from memory.
- **Scheduling by gut.** Coverage is built from "we usually need two servers on Friday" rather than from forecasted covers and explicit staffing rules, and it routinely violates availability or pushes people into overtime.
- **No institutional memory.** When the operator is out, decisions get made without the context the owner carries in their head.

The thesis here is that an AI assistant is the right interface for this user — but *only* if it is given real, structured, restaurant-specific context. A generic chatbot that says "consider reviewing your food costs" is worthless. Sage is built so that it can say "your chicken cost is up 12% and your Caesar salad is running 14% over its theoretical chicken portion — fix the portioning before you touch the menu price."

SECTION 03

System Architecture & Stack

The product is split into three deployable pieces plus a managed data layer.

BACKEND

FastAPI on Railway

A FastAPI application hosting the entire API surface: the Anthropic proxy, all AI endpoints, integration sync jobs, reporting endpoints, and a background scheduler.

DATA LAYER

Supabase Postgres

Postgres, magic-link auth, file storage, and — critically — Row-Level Security as the multi-tenant isolation boundary. Every tenant-scoped table keyed on `auth.uid()`.

FRONTEND

Vanilla JS PWA on Vercel

No frontend framework. Standalone HTML pages with shared ES modules for auth, location switching, the Sage chat widget, and the service worker.

AI LAYER

Anthropic Claude API

Cost-aware model selection between `claude-sonnet-4-6` as the workhorse and `claude-haiku-4-5` for higher-volume calls. Every call logged and costed.

Backend characteristics

- `AsyncIOScheduler` runs recurring jobs in-process: proactive alert checks, scheduled review pulls, and the weekly restaurant-memory rebuild.
- `httpx.AsyncClient` is used for all outbound calls (Anthropic, Supabase REST, Outscraper) so the event loop is never blocked on I/O.
- **Auth is JWT verification against Supabase.** Every protected endpoint depends on `get_current_user`, which extracts the Bearer token and verifies it against Supabase. Token verification results are cached in-process for five minutes behind a lock, with pruning above 2,000 entries — this removes a blocking HTTP round-trip from the hot path under concurrent load, which was the dominant latency source.

Frontend behavior

The UI is a set of standalone HTML pages served as static files from Vercel. Shared behavior lives in plain ES modules — `config.js`, `auth.js`, `location-switcher.js`, `sage-chat.js`, and the PWA service worker.

The service worker precaches core assets, serves them offline, and deliberately **bypasses the cache for any Supabase or backend request** so API calls always hit the network. This was a deliberate choice: caching API responses in a service worker is a common source of "why is my data stale" bugs, and for an operations tool stale data is worse than a spinner.

AI layer notes

All model calls go to `https://api.anthropic.com/v1/messages`. The backend maintains a token-cost table (per-million-token input/output rates for each model) and a `log_usage` helper that records every Anthropic call to a `usage_log` table with the raw response JSON, computed cost, and per-call metadata. AI spend is a real line item, so it is measured per user, per feature. The `/claude` endpoint is a thin authenticated proxy so the Anthropic key never reaches the browser.

> **DESIGN NOTE**
>
> Keeping the backend as a single FastAPI app is a deliberate solo-engineering choice — it optimizes for one person being able to hold the whole system in their head. The natural split point is when a second engineer joins; until then, a monolith with clean internal boundaries is the right tool.

SECTION 04

AI Feature Deep Dives

The interesting engineering is in how each AI feature is *grounded*.

The pattern throughout is the same: gather structured data from Postgres, assemble it into a tightly-scoped context block, hand Claude a system prompt with strict output rules, and post-process the result against business logic. Below, each feature is described as built.

4.1 Sage — the conversational operations assistant

Sage is the front door. The chat widget is present on every major page and posts the operator's message, the recent conversation history, and a `page_context` string indicating what page they're on.

The endpoint does *not* just forward the message. Before calling Claude it assembles a deep context payload for the current tenant:

1. **Restaurant memory** — the synthesized intelligence profile: `ai_context_summary`, `identity`, `operator_profile`, `learned_patterns`, `seasonal_patterns`, `economic_model`.
2. **Operator model** — the person behind the restaurant (see 4.2).
3. **Latest ops brief** — the most recent Sage Daily Brief headline and `today_focus`.
4. **Active proactive alerts** — up to three non-dismissed alerts.
5. **Recent decisions** — what the operator recently actioned vs. skipped, from `decision_actions`.
6. **Seasonal context for today** — if today's day-of-week multiplier deviates ≥10% from average, that is stated explicitly.
7. **High-confidence learned patterns** — patterns with confidence ≥ 0.7.
8. **Page hint** — a sentence describing what the operator is looking at right now.

These compose into a system prompt that opens "*You are Sage, the AI operations expert for {restaurant name}. You know this restaurant deeply. Here is everything you know:*" followed by the assembled context. The prompt also encodes personality and guardrails: be specific not generic, tie every answer back to known data, admit when data is missing, no AI disclaimers, redirect off-topic questions.

The conversation history is truncated to the last 10 exchanges. The Claude call uses `claude-sonnet-4-6` with a 600-token budget in advisor mode. Every call is logged via `log_usage`.

> **INTERESTING DECISION**
>
> The context assembly is intentionally conditional — empty sections are dropped, not sent as "no data." This keeps the prompt tight, keeps token cost down, and avoids teaching the model that "no data" is a normal state it can echo back to the user.

4.2 Operator Doppelgänger & the Standards Interview

This is the most distinctive feature in the product. The premise: Sage should be able to answer *as the owner*, not just as a generic advisor — so that when the owner is off-site, their management team gets guidance in the owner's actual voice and standards.

The **Standards Interview** is a 10-question wizard. It captures the operator's hard numbers and soft style: food cost red line and amber line, labor cost red line, communication style (`data-first` / `direct` / `detailed` / `formal`), a voice sample (how they actually write), a sign-off phrase, an opening phrase, phrases they would never say, and explicit operating rules.

The interview endpoint upserts the `operator_model` row and creates interview-sourced rows in `operator_rules` (each rule typed as `never_do` / `always_do` with a confidence score). The owner can later correct what the doppelganger said via a correction endpoint, which logs to `doppelganger_corrections` and increments a count on the model — a feedback loop for future refinement.

Doppelganger mode is triggered by a mode flag on the chat endpoint. When set, the system prompt is *completely different*. It does not say "you are Sage." It says:

> *You are not Sage. You are the owner of {restaurant name}, speaking directly to your management team.*

It then injects the owner's voice sample, never-say phrases, sign-off, and opening phrase; maps their communication style to a concrete instruction; turns their thresholds into stakes; lists their non-negotiable rules; and supplies the current restaurant situation. The response rules force first person ("I want…", "Here's what I need from you…"), cap length at four sentences, and explicitly forbid saying "As the owner" — "*just speak. You are the owner.*"

INTERESTING TRADEOFF

This feature is powerful and a little uncanny, so it is gated behind an explicit mode flag and a structured interview rather than being inferred. The owner opts in, the owner provides the voice, and the owner can correct it. The correction log is the path from "good impression" to "actually accurate."

4.3 Restaurant Memory — the synthesized intelligence layer

This is the substrate every other AI feature reads from. `build_restaurant_memory()` runs weekly via the scheduler, can be triggered manually, and is scoped per location. It pulls a wide cross-section of the tenant's data — POS items, inventory, price history, sentiment, decisions, events, depletion, sales, vendors, invoices, product catalog — and synthesizes it into a structured profile with these components:

- **Identity** — top sellers, menu item count, estimated average check, average daily covers, peak days and slow days (derived by bucketing sales history covers by day name).
- **Operator profile** — decision acceptance rate, what decision types the operator acts on vs. ignores, and an estimated "brief check hour" inferred from the timestamps on actioned decisions.
- **Learned patterns** — a list of observations, each with a confidence score, `times_confirmed` count, `observed_since` date, and data source. Confidence is reinforced over time: when a pattern is re-observed in a later run, its confidence is

incremented (capped at 0.9) rather than reset. Patterns are sorted by confidence and capped at 30.

- **Economic model** — top cost drivers by spend, revenue share by menu category.
- **Causal relationships** — candidate cause→effect links. For example, if protein prices moved >8% in the same window food-cost pressure appeared, a `protein_cost_food_cost` relationship is created at 0.55 confidence and reinforced on subsequent observation.
- **Seasonal patterns** — day-of-week multipliers, with peak/slow flags and a `sample_weeks` count so confidence is visible.
- **ai_context_summary** — a natural-language paragraph assembled from the structured parts, which is what gets injected verbatim into Sage's and the Daily Brief's system prompts.

> **INTERESTING DECISION**
>
> The memory builder is *deterministic Python*, not an LLM summarization step. Pattern detection, confidence reinforcement, and the summary assembly are all code. This was deliberate — the memory layer is the ground truth Sage reasons from, so it has to be reproducible, debuggable, and free of hallucination. The LLM consumes the memory; it does not author it. Each run is also *existing-memory-aware*: it merges into the prior profile so confidence and observation counts accumulate across weeks rather than starting fresh.

4.4 AI Inventory Scanning — Claude vision

The operator photographs a storage shelf; Claude vision (`claude-sonnet-4-6` with an image block) identifies and counts the packaged items. The naive version of this is "send the photo, ask what's on the shelf." This version is heavily grounded:

- **Catalog context.** The tenant's product catalog (location-scoped, falling back to global) is loaded — names, SKUs, brands, container types, visual cues, typical colors — and formatted into the prompt as "Known items in this restaurant's catalog." The model is instructed to match to existing SKUs and only invent a new item when nothing matches.
- **Shelf profiles.** If a shelf profile ID is supplied, the expected SKUs for that physical shelf are injected ("This shelf typically contains…").

- **Correction memory.** This is the clever part. Past human corrections are loaded and aggregated per item into an average count offset. If an item has historically been undercounted by ≥1 unit on average, the prompt tells the model so: "*historically undercounted by 1.4 units on average — count more carefully (based on 3 past corrections).*" Name corrections are also fed back: "*When you see 'X', it is actually 'Y' in this restaurant.*"

The system prompt enforces strict scope (packaged items only — no produce, no open containers, no equipment), explicit confidence thresholds (0.90+ = very clear, down to <0.50 = flag for manual entry), and a strict JSON output schema including per-item `confidence`, `confidence_level`, `catalog_match`, `needs_review`, and `correction_applied` flags. It also asks the model to return *visual cue suggestions* — descriptions of what it visually used to identify each item — which can be written back to the catalog to make *future* scans better.

> **KEY INSIGHT**
>
> This is a closed learning loop built without any model fine-tuning. Accuracy improves purely through prompt-injected, restaurant-specific feedback.

4.5 Forecasting — covers and revenue projection

The forecast — `buildForecast(salesHistory, bookedEvents, multipliers, daysAhead)` — runs **client-side**. It projects covers and revenue forward (default 14 days) from sales history:

1. Sales history is bucketed by day-of-week.
2. For each future date, a base covers figure is computed as a **weighted average** of recent same-day-of-week history (more recent weeks weighted more heavily).
3. A **seasonal multiplier** for that day is applied, loaded from a `seasonal_multipliers` table.
4. **Booked events** for that date are added on top — their `guest_count` is added directly to projected covers.
5. Revenue is derived from projected covers and a blended average check (computed from history rows that have both covers and revenue).
6. Each projected day carries a **confidence** value reflecting how much same-day-of-week history backed it.

INTERESTING DECISION

Forecasting lives in the browser. It is pure arithmetic over data the client already has loaded for the forecast page, so there is no reason to pay a network round-trip or server compute for it. The seasonal multipliers it consumes, however, are derived server-side by `build_restaurant_memory` — so the heavy, accumulating analysis is centralized and the light, interactive projection is local. This split keeps the forecast page instant and responsive while the "intelligence" stays in one auditable place.

4.6 Autofill Scheduler — constraint-satisfaction staffing

This is the most algorithmically dense piece of frontend code in the product. It auto-builds a weekly staff schedule that respects a real set of constraints — no LLM involved, because scheduling is a constraint-satisfaction problem and a model would be both slower and less trustworthy than explicit logic.

The constraint set:

- **Coverage rules** — `staffing_rules` define a minimum (and optional maximum) head count per role per daypart. The effective-rules function merges the operator's explicit rules with sensible defaults — but only for roles that actually have employees and have no explicit rule.
- **Day availability** — each employee's recurring weekly availability.
- **Daypart availability** — employees declare which shift types they will work; candidates whose available dayparts exclude the daypart are filtered out.
- **No double-booking** — the same employee cannot be placed on the same daypart twice in one day (different dayparts on the same day *are* allowed).
- **Max-hours caps** — each employee's `max_weekly_hours` is a hard ceiling; a candidate shift that would breach it is rejected.
- **Role matching** — candidates must match the role the coverage rule needs, with `role_override` applied when an employee is filling outside their default role.

The algorithm runs in two phases:

Phase A — fill coverage gaps. It loops: `computeSuggestions()` returns every open role/daypart/date gap with a ranked candidate list (sorted by current weekly hours ascending, so the least-loaded eligible person is placed first). It places the single best candidate for the top gap, **pushes that shift into the in-memory schedule**, and re-runs. Because each placement updates the in-memory state, the next iteration sees reduced need, updated weekly hours, and the new double-booking constraint. A guard counter caps iterations at 400.

Phase B — fill toward each employee's max hours. After coverage is satisfied, it ranks employees by how far they are below their cap (most underfilled first, for a balanced fill), and places additional shifts while honoring day availability, daypart availability, the no-same-daypart-twice rule, the max-hours cap, *and* coverage `max_count` so a role/daypart is never overstaffed past its rule.

> **INTERESTING DECISION**
>
> The greedy, iterative-with-in-memory-mutation approach was chosen over a global optimizer because it is debuggable and explainable to the operator — every placement is the obvious "best available person for the most urgent gap right now." A schedule the operator does not understand is a schedule they will not trust, and the entire output is editable afterward.

4.7 Review Sentiment Analysis

Reviews are pulled from Google via the Outscraper API. Sentiment analysis is intentionally **two-tier**:

- **analyze_sentiment_from_ratings** — a fast, deterministic scorer that maps star ratings to a 0–10 score and a Positive/Mixed/Negative label. No model call. This powers the monthly sentiment chart and the historical 12-month backfill, because it is reliable and free, and ratings-only reviews (no written text) still count toward it.
- **Claude analysis (claude-sonnet-4-6)** — for the qualitative layer. Written review text is passed to Claude with a strict-JSON system prompt that returns an overall label, a 0–10 score, a 2–3 sentence summary, and 3–5 structured *positives* and *negatives* (each a theme + detail). This feeds Sage's understanding of "customers love X, need work on Y."

Results are written to `sentiment_history` (monthly snapshots) so trend charts and the restaurant memory's sentiment-trend pattern detection both have a time series to work from.

> **INTERESTING DECISION**
>
> Not everything needs an LLM. Star ratings are already a clean signal; using a model to "analyze" them would add cost, latency, and a hallucination surface for no gain. Claude is reserved for the part it is uniquely good at — extracting themes from unstructured text.

4.8 Proactive Alerts

This is a rules engine, not a model. It scans the tenant's data for conditions worth interrupting the operator over and writes them to `proactive_alerts`. Examples implemented:

- **Depletion-rate alerts** — for each catalog item it compares the latest daily usage against the historical average; if usage is running 40%+ above normal *and* the projected days-of-stock-left is under four, it fires an alert with severity scaled by urgency ("Using 8.2 units/day vs normal 5.1 — ~2 days left").
- **Reorder-point alerts** — quantity on hand at or below the catalog `reorder_point`.
- **Price-change and other operational conditions** follow the same pattern.

The save-alert helper **deduplicates** — it checks for an existing open alert of the same type before inserting, so the operator is not buried in repeats of the same condition. Alerts carry an action URL and label so they link straight to the page where the operator can act, and the active set is injected into Sage's context so the assistant is aware of what the system has already flagged.

4.9 Invoice Processing

Supplier invoices arrive two ways. The EDI path receives **EDI 810 (invoice)** and **EDI 855 (PO acknowledgment)** transactions from middleware (SPS Commerce, TrueCommerce, Orderful), parses them, matches them to existing purchase orders by

PO number, and explodes them into `invoice_line_items` rows. EDI 855s update PO status and record line-level discrepancies. There is also an AI-assisted path for parsing uploaded invoice documents into the same line-item structure.

Line items matter because they are the raw material for the rest of the intelligence layer: per-ingredient cost over time feeds `price_history`, vendor spend rollups feed the restaurant memory's vendor intelligence, and price movements feed both the Daily Brief and proactive alerts. The Daily Brief specifically cross-references price spikes against over-portioning variance — if an ingredient shows up in *both*, the prompt instructs Sage to diagnose portioning before recommending a price increase, because the cheaper internal fix is almost always the right first lever.

4.10 The Sage Daily Brief

Worth calling out as the synthesis of everything above. The brief-generation endpoint gathers top/low sellers, items needing reorder, price alerts, over-portioning variance alerts, review sentiment, upcoming events, waste and comp data, distributor rebates, and menu-engineering classifications (Stars, Plowhorses) — assembles them into a context block — layers in the restaurant memory, learned patterns, operator preferences, and a *learned recommendation strategy* — and asks `claude-sonnet-4-6` to return a strict-JSON brief: a headline, and categorized `orders` / `prep` / `menu_actions` / `alerts` arrays plus a single `today_focus`.

The system prompt is unusually strict because LLMs love to drift into vague advice. It hard-defines each category ("orders = ONLY specific ingredients to purchase from a supplier — never a dish name"), forbids invented categories, caps items per section, and encodes a **decision-priority hierarchy** for food-cost issues: fix portioning first (free margin), then waste, then recipe, and raise menu price only as a last resort. After Claude responds, a filter removes anything the operator handled or skipped recently, and high-confidence portioning variance alerts that Claude omitted are *pinned* back in by code.

> *The model proposes; business logic disposes.*

SECTION 05

Data Model & Multi-Tenant Security

The product is multi-tenant, and the entire isolation model rests on *Postgres Row-Level Security*. This is the part of the system treated as zero-tolerance.

The RLS pattern

Every tenant-scoped table has RLS enabled and a policy keyed on the authenticated user. For tables that carry `user_id` directly, the policy is the direct form:

```
alter table employees enable row level security;
create policy "Users own employees" on employees
  for all using (auth.uid() = user_id);
```

For child tables that do *not* carry `user_id` — they reference a parent — the policy verifies ownership *through* the parent:

```
create policy "Users own availability" on employee_availability
  for all using (
    employee_id in (select id from employees where user_id = auth.uid())
  );
```

This subquery-through-parent pattern is applied consistently — availability through employees, shifts through schedules, and so on — so a row is reachable only if its ownership chain terminates at the requesting user. Policies are declared inline with the table schema, which keeps the policy next to the structure it protects.

The core tables

The data model spans operations end to end:

- **Scheduling** — employees, employee_availability, shift_definitions, schedules, schedule_shifts, staffing_rules, labor_targets.
- **Inventory & costing** — product_catalog, inventory_items, inventory_scans, inventory_scan_corrections, shelf_profiles, inventory_depletion.
- **Suppliers & invoices** — vendors, invoices, invoice_line_items, purchase_orders, price_history, rebates.
- **Sales & forecasting** — pos_items, sales_history, seasonal_multipliers, booked_events.
- **Reviews** — sentiment_history, plus monthly review snapshots.
- **The AI layer** — restaurant_memory (the synthesized profile, with confidence-scored components), operator_model and operator_rules (the doppelganger), doppelganger_corrections, ops_briefs, proactive_alerts, decision_actions, decision_learning, usage_log.

Defense in depth

RLS is the boundary, but the backend does not rely on it alone:

- **The backend uses the Supabase service-role key**, which *bypasses* RLS — so service-role endpoints must, and do, verify ownership at the application layer. The frontend-supplied `user_id` is never trusted; the authenticated identity comes from `get_current_user`'s JWT verification, and queries are scoped using the user ID derived from the verified token.
- **Location scoping** is layered on top of user scoping. The product is multi-location; queries take a `location_id` and filter so location-specific data and shared global data both resolve correctly without leaking across locations.
- **The Anthropic key is server-side only** — the proxy endpoint exists specifically so the key never ships to the browser.
- **The service worker never caches API responses**, so there is no path for one user's data to be served from cache to another on a shared device.

SECTION 06

Testing & Deployment

Deployment topology

- **Frontend → Vercel.** A preview workflow deploys a Vercel preview on every push to a non-main branch. The main deploy workflow deploys to production on push to main.
- **Backend → Railway.** The FastAPI app runs on Railway, built from a Dockerfile.

Post-deploy smoke checks

A smoke-check job runs after every production deploy:

- **Backend health check** against the API root, with retry-up-to-3-times logic because Railway can cold-start — a single failed curl is not treated as a real failure until three attempts spaced 10s apart all fail with a 5xx.
- **Frontend asset checks** — it curls a list of critical static assets on the production domain and fails the job if any returns anything other than 200.
- **A failure annotation** points at the production URL and Railway logs.

This catches the most common real-world deploy failures — a broken backend boot, or a missing/renamed static asset — within seconds of the deploy completing, which is exactly when you want to know.

The intended layered test strategy

The project's engineering standards define a four-layer testing stack: a **Playwright** end-to-end suite using user-facing locators, run as a fast read-only pre-push hook (page loads, navigation, security headers, API health — no DB writes, target ≤90s) and as a full suite in CI against Vercel previews; **Lighthouse CI** performance and quality budgets on previews; **visual regression** via Percy; and **Sentry** for production error tracking with releases tied to deploys. The smoke job and asset checks already running in the deploy workflow are the post-deploy layer of that strategy.

SECTION 07

Closing Reflection

What was hard, and what I learned.

01 Grounding is the whole game.

The hard part of building an AI product is not calling the model — it is everything around the call. Every Sage feature spends far more code on assembling context and post-processing output than on the API request itself. The Daily Brief endpoint is hundreds of lines of data gathering and a strict prompt; the actual Claude call is a dozen lines. The lesson that compounded across the project: an LLM is only as good as the structured context you can hand it, and the engineering value is in building that context reliably.

02 Decide deliberately where intelligence lives.

I kept coming back to the same question — should this be a model call, deterministic code, or client-side arithmetic? The answers ended up principled: pattern detection and the memory layer are deterministic Python because they are the ground truth and must be reproducible; the autofill scheduler is a constraint solver because scheduling is a CSP and an explainable greedy algorithm earns more operator trust than a black box; forecasting is client-side because it is light arithmetic over already-loaded data; star-rating sentiment is a lookup table because the signal is already clean. Claude is reserved for what it is uniquely good at — reasoning over messy, restaurant-specific context and extracting themes from unstructured text. Reaching for the model everywhere would have made the product slower, more expensive, and less trustworthy.

03 Closed feedback loops without fine-tuning.

Two features — inventory scan correction memory and the doppelganger correction log — get measurably better over time purely by feeding human corrections back into the next prompt. No training pipeline, no fine-tuning. For a solo project this was the right call: it is cheap, debuggable, and immediate.

04 Multi-tenancy is a discipline, not a feature.

RLS keyed on `auth.uid()` is the boundary, but the backend runs on the service-role key that bypasses it — so "the database will protect us" is not enough. Every service-role endpoint verifies ownership at the application layer, the frontend-supplied user ID is never trusted, location scoping is layered on top of user scoping, and the service worker is deliberately barred from caching API responses. Getting this right meant treating data isolation as something to prove, not assume.

05 The unglamorous infrastructure decisions mattered most.

Caching JWT verification for five minutes removed the dominant latency source under load. Bypassing the service-worker cache for API calls eliminated a whole class of stale-data bugs. Retry logic in the post-deploy smoke check stopped cold-starts from creating false alarms. None of these are interesting in a demo, and all of them are the difference between a prototype and something an operator can run their business on.

> *If I were to summarize this in one line: it is an AI product where the AI is the smallest part of the system, and that is on purpose.*

— END —